Tech Challenge – Fase 3
JavaEats_Fase3 – Relatório Técnico

Arquitetura e Desenvolvimento Java - FIAP
Data de entrega: 23/03/2026

Autores:

- Everton Nóbrega Barbosa – RM 368760
- Irving Lucas da Silva Melo – RM 367546
- Lucas Novaes Vieira – RM 367795
- Luis Fernando Silva do Nascimento – RM 367853

Repositório:
- https://github.com/nbeverton/fiap-tech-challenge-fase3

Vídeo:
- https://www.youtube.com/watch?v=Va5EO5NqMoc

Sumário

Autores do projeto
Link para repositório
Link para o vídeo explicativo

1. Introdução

2. Descrição detalhada do projeto

3. Objetivo do projeto

4. Arquitetura do sistema
    4.1 Stack e decisões de arquitetura
    4.2 Visão geral por componentes
    4.3 Organização em camadas

5. Requisitos funcionais do sistema
    5.1 Gerenciamento de usuários
    5.2 Gerenciamento de pedidos
    5.3 Gerenciamento de pagamentos

6. Requisitos não funcionais
    6.1 Arquitetura em Múltiplos Serviços
    6.2 Segurança com Spring Security + JWT
    6.3 Comunicação Assíncrona com Kafka
    6.4 Resiliência (Resilience4j)
    6.5 Boas práticas de arquitetura

7. Configuração do sistema
    7.1 Pré requisitos
    7.2 Execução
    7.3 Encerrar execução
    7.4 Observação sobre rebuild

8. Execução do sistema

1. Introdução

Este relatório técnico tem como objetivo documentar o desenvolvimento do sistema JavaEats_Fase3, apresentando sua proposta, arquitetura, *endpoints* da API e instruções para configuração e execução. O documento segue os padrões acadêmicos estabelecidos pela Associação Brasileira de Normas Técnicas (ABNT), visando clareza, organização e padronização das informações apresentadas.

2. Descrição detalhada do projeto

Um grupo de restaurantes decidiu contratar estudantes para construir um sistema de gestão compartilhada para seus estabelecimentos. Essa decisão foi motivada pelo alto custo de sistemas individuais, o que levou os restaurantes a se unirem para desenvolver uma solução única e centralizada.

O sistema permite que os clientes escolham restaurantes com base na comida oferecida, e não na qualidade ou complexidade do sistema de gestão utilizado, promovendo uma experiência mais justa e padronizada entre os estabelecimentos participantes.

3. Objetivo do projeto

O objetivo é criar um sistema de pedidos online para um restaurante, que permita criar pedidos, processar pagamentos e controlar o status dos pedidos. O sistema deve ser resiliente e seguro, capaz de lidar com falhas e *timeouts*. Na descrição de cada um dos requisitos há informações adicionais relativas a cada um dos itens.

4. Arquitetura do sistema

4.1. Stack e decisões de arquitetura

O sistema JavaEats – Fase 3 foi desenvolvido em Java 21 com Spring Boot, adotando uma arquitetura organizada em camadas e orientada a casos de uso, inspirada na *Clean Architecture* / Hexagonal. O objetivo foi manter as regras de negócio isoladas de detalhes técnicos (*frameworks*, banco e integrações), facilitando manutenção, testes e evolução do projeto.

A persistência foi implementada com Spring Data MongoDB, utilizando MongoDB como banco de dados. O *build* e empacotamento são feitos via Maven, gerando um JAR executável. Para execução padronizada em diferentes ambientes, foi utilizado Docker Compose, permitindo subir rapidamente a aplicação, o banco e o simulador de pagamento externo em um único comando.

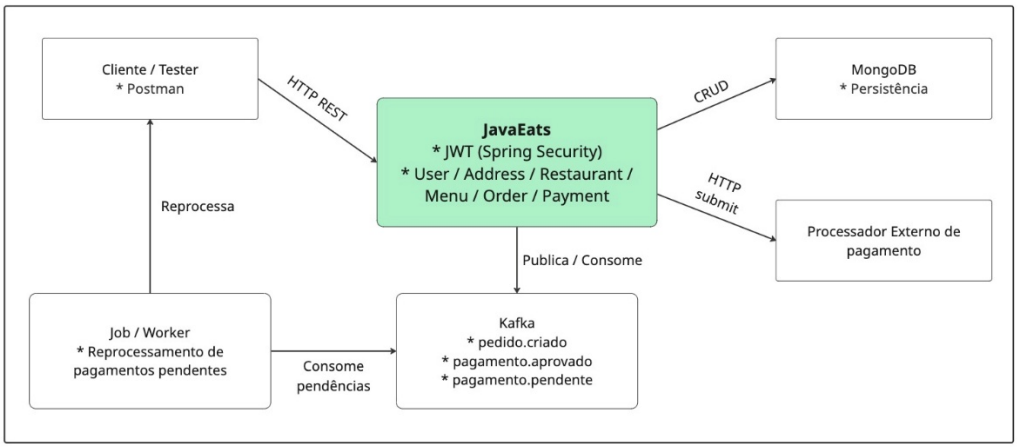
Para robustez no processamento de pagamentos (serviço externo eventualmente disponível), foram utilizados mecanismos de resiliência com Resilience4j (*circuit breaker* e *timeout*) e um fluxo de reprocessamento automático quando necessário.

4.2. Visão geral por componentes

A solução é composta por uma API REST em Spring Boot que concentra o domínio do sistema (usuários, restaurantes, menus, pedidos e pagamentos) e integra-se a componentes externos essenciais: MongoDB para

persistência, Kafka para eventos assíncronos e um processador externo de pagamentos (procpag) fornecido pelo professor. Quando um pagamento é criado, o sistema tenta processá-lo através do serviço externo, em caso de indisponibilidade, o sistema mantém o estado consistente e encaminha o pagamento para reprocessamento automático (Figura 1).

Figura 1. Diagrama de componentes (visão geral)



Os principais componentes da aplicação:

- API JavaEats (Spring Boot): expõe *endpoints* REST, aplica validações e regras de negócio e orquestra o fluxo de pedidos e pagamentos.
- MongoDB: persistência dos dados (usuários, pedidos, pagamentos, restaurantes, menus e endereços).
- Kafka: barramento de eventos para desacoplamento do fluxo (ex.: pedido.criado, pagamento.aprovado, pagamento.pendente).
- Processador Externo (procpag): simula um serviço de pagamento eventualmente disponível, utilizado para autorizar/confirmar pagamentos.
- Job/Worker de reprocessamento: responsável por tentar novamente pagamentos pendentes em intervalos configuráveis, permitindo que o sistema se recupere automaticamente quando o serviço externo voltar a funcionar.

4.3. Organização em camadas

A aplicação foi estruturada em camadas com responsabilidades bem definidas:

- Core (Domínio + Casos de Uso): onde ficam as entidades, regras e transições de estado (ex.: evolução do pedido), além dos casos de uso que coordenam o fluxo da aplicação. Essa camada não depende de banco, web ou frameworks específicos, e se comunica com o "mundo externo" por interfaces (*ports*).
- Infraestrutura de Persistência: implementa os *adapters* que conectam os *ports* do Core ao MongoDB via Spring Data. Inclui documentos, repositórios e mapeadores responsáveis por transformar modelos do domínio em representações adequadas ao armazenamento.
- Infraestrutura Web (API REST): contém *controllers* e DTOs, responsáveis por receber requisições HTTP, validar entrada, traduzir dados para comandos/casos de uso e devolver respostas padronizadas.
- Infraestrutura de Configuração: reúne os *beans* e o "wiring" do Spring, conectando casos de uso aos *adapters* e configurando componentes transversais (ex.: Swagger, integrações e jobs). Essa camada não contém regra de negócio, apenas orquestra dependências.

5. Requisitos funcionais do sistema

O projeto atende integralmente aos requisitos funcionais descritos no documento do Tech Challenge – Fase 3. A seguir, os requisitos são apresentados de forma textual, descrevendo o comportamento do sistema e como cada etapa é executada na prática.

5.1 Gerenciamento de usuários

Aqui é possível criar um novo usuário do tipo cliente passando no corpo da requisição o userType do tipo CLIENT e as informações básica. Após criar o cliente, é possível fazer uma nova requisição para realizar a sua autenticação, com isso é criada uma chave Token pessoal e que será passada pelo sistema para a liberação das demais requisições do usuário.

Além do usuário do tipo CLIENT, o sistema também permite a criação de usuários do tipo OWNER (dono de restaurante) e ADMIN (administrador do sistema), cada um com suas permissões específicas. O OWNER consegue consultar pedidos do seu próprio restaurante, além de aceitar ou recusar um pedido e seguir o fluxo para entrega até o cliente. Já o ADMIN tem credenciais de administrador de sistema, e por isso, consegue visualizar os pedidos e rejeitar ou aceitar, bem como visualizar os pedidos de todos os clientes.

A tabela 1 abaixo mostra o método, um exemplo de corpo de requisição para a criação de um usuário do tipo CLIENT e a descrição da requisição.

Tabela 1 – Exemplo de requisição de criação e autenticação de usuário.

Endpoint	Método	Corpo da requisição	Autenticação	Descrição

POST	/users	{
		"name": "Usuário client",
		"userType": "CLIENT",
		"email": "userTech.client@example.com",
		"login": "client.techChallenge",
		"password": "SenhaForte123!",
		"addressId": "{{addressIdClient}}",
		"addressType": "HOME",
		"label": "casa"
		}
			Nenhuma autenticação necessária	Realiza o cadastro de um novo usuário do tipo cliente

POST	/auth/client	{
		"login": "client.techChallenge",
		"password": "SenhaForte123!"
		}
			Criação do token de autorização pessoal	Autoriza o cliente e cria uma nova chave token de acesso

5.2 Gerenciamento de pedidos

O sistema permite a criação de pedidos por um cliente autenticado. Para isso, o cliente envia o token JWT no campo de autorização e informa no corpo da requisição os dados necessários para compor o pedido, incluindo o restaurante, o endereço de entrega e os itens (menus) com suas quantidades.

Após o pedido ser criado, o sistema calcula automaticamente o valor total, registra o pedido e retorna o ID gerado, que passa a ser utilizado nas etapas seguintes do fluxo. Em seguida, o pedido pode ser aceito por um usuário com perfil OWNER (dono do restaurante) ou ADMIN, respeitando as regras de autorização do sistema, isto é, o OWNER só consegue gerenciar pedidos do próprio restaurante.

Além da criação e aceitação, o sistema disponibiliza consulta de pedidos vinculados ao cliente autenticado (filtrando automaticamente pelo token) e também a consulta de um pedido específico via id. A Tabela 2 apresenta um exemplo de criação de pedido, e a Tabela 3 exemplifica as requisições de consulta.

Tabela 2 - Exemplo de requisição de criação e autenticação de usuário

Endpoint	Método	Corpo da requisição	Autenticação	Descrição

POST	/orders	{
		"restaurantId": "{{restaurantId}}",
		"userAddressId": "{{userAddress}}",
		"items": [
		{
		"menuId": "{{menuId}}",
		"quantity": 1
		}
		]
		}
			Authorization: {{tokenClient}};	Cria um novo pedido passando no corpo da requisição:
				* Id do restaurante;
				* Id do vínculo endereço/usuário
				* Itens do pedido (Id do menu e quantidade)

Ajuste importante aplicado no projeto: as rotas que dependem do usuário (ex.: listar pedidos do cliente) não precisam receber userId manualmente, porque o sistema obtém esse vínculo a partir do token, reduzindo risco de inconsistência e aumentando a segurança.

Tabela 3 – Exemplo de requisição de consulta de pedidos.

Endpoint	Método	Corpo da requisição	Autenticação	Descrição
GET	/orders /{{orderId}}	N/A	Authorization: {{tokenClient}};	Consulta o status de um pedido passando o Id do pedido.
GET	/orders/me	N/A	Authorization: {{tokenClient}};	Consulta o pedido passando o Token de acesso de um cliente.

5.3 Gerenciamento de pagamentos

O pagamento foi implementado com integração ao serviço externo fornecido pelo professor (processamento-pagamento-externo, via imagem Docker), que simula um processador de pagamentos eventualmente disponível. Quando uma requisição de pagamento é criada, o sistema registra o pagamento como PENDING e aciona o serviço externo para processar a transação. Se o serviço estiver disponível e autorizar o pagamento, o pagamento é confirmado e seu status passa para PAID. Nesse momento, o sistema também registra informações relevantes do processamento, como provider e transactionId, e atualiza automaticamente o status do pedido para permitir a continuidade do fluxo (preparo, saída para entrega e entrega).

Se o serviço externo estiver indisponível, o sistema não altera o status do pagamento para FAILED. Em vez disso, o pagamento permanece como pendente e o pedido é mantido em um estado consistente para aguardar reprocessamento. Esse reprocessamento é feito automaticamente pelo *retry* em uma fila de mensageria utilizando o Kafka, onde a cada 30 segundos ele chama as mensagens pendentes. Quando o processador externo volta a responder, o pagamento é submetido novamente e, se aprovado, passa para PAID, atualizando o pedido automaticamente para seguir o fluxo normal.

A Tabela 4 apresenta um exemplo de requisição de criação de pagamento, utilizando o id do pedido na URL e informando no corpo o valor e o método de pagamento.

Tabela 4 – Exemplo de requisição de criação de pagamento.

Endpoint	Método	Corpo da requisição	Autenticação	Descrição

POST	/orders /{{orderId}} /payments	{
		"amount": 34.90,
		"method": "PIX"
		}
			Authorization: {{tokenOwner}};
			ou
			Authorization: {{tokenAdmin}};
				Cria um pagamento passando o Id do pedido no endereço e valor e método de pagamento no corpoda requisição.

Base URL (execução padrão em Docker): http://localhost:8081/

6. Requisitos não funcionais

Os requisitos não funcionais definidos no enunciado foram atendidos por meio de decisões de arquitetura, segurança, comunicação e resiliência que tornam o sistema mais consistente, escalável e adequado para execução em ambientes diferentes (local e Docker). A seguir, descrevemos como esses requisitos foram implementados e como eles impactam o funcionamento do projeto.

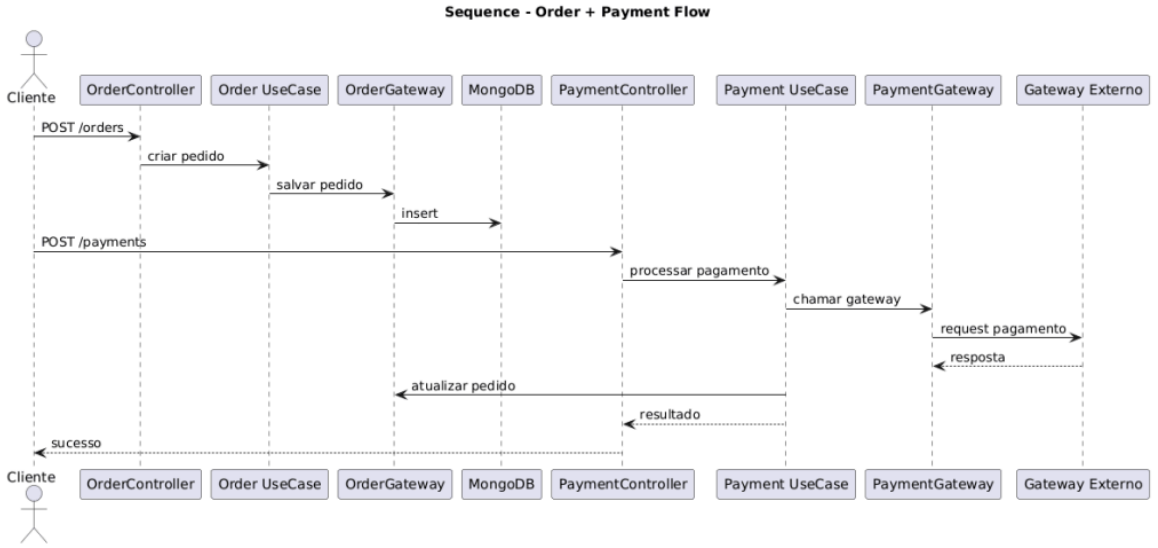
6.1 Arquitetura em Múltiplos Serviços

A solução foi estruturada com separação clara de responsabilidades, adotando uma arquitetura baseada em serviços e componentes independentes. O sistema principal (responsável por regras de negócio, pedidos e orquestração do fluxo) executa como uma aplicação Spring Boot e integra-se a serviços externos fundamentais para o funcionamento da plataforma. Entre esses serviços, destacam-se o banco de dados MongoDB, o processador externo de pagamentos fornecido pelo professor via imagem Docker (procpag) e o serviço de comunicação assíncrona com o Kafka.

Do ponto de vista funcional, essa separação permite que o fluxo de pedidos continue operando mesmo quando serviços externos estejam instáveis, além de facilitar a execução do ambiente em Docker Compose com todos os componentes necessários. Essa abordagem também melhora a manutenção do sistema: alterações no processamento de pagamento, por exemplo, tendem a afetar somente o módulo de integração, sem comprometer o núcleo do domínio de pedidos.

A figura 2 a seguir ilustra o fluxo completo de criação de um pedido por um usuário do sistema JavaEats, desde a criação de um novo pedido (POST orders) até o processamento do pagamento externo.

Figura 2. Fluxo de pagamento e pedido



6.2 Segurança com Spring Security + JWT

A segurança foi implementada com Spring Security e autenticação baseada em JWT. O sistema disponibiliza *endpoints* de autenticação no prefixo /auth, permitindo que usuários se autentiquem e obtenham um token JWT.

Esse token é utilizado nas requisições subsequentes por meio do *header* Authorization: Bearer <token>, habilitando o acesso às rotas protegidas conforme o perfil do usuário.

A autorização segue o modelo de perfis/roles (CLIENT, OWNER, ADMIN), garantindo que cada tipo de usuário execute apenas operações compatíveis com suas permissões. Além disso, informações sensíveis de identificação (como o userId) são derivadas do token autenticado no *backend*, reduzindo a dependência de dados enviados no corpo da requisição e evitando inconsistências ou tentativas de manipulação. Dessa forma, o cliente só consegue criar pedidos, consultar seus próprios registros e realizar ações permitidas após autenticação válida, enquanto ações administrativas e de gestão do restaurante exigem perfis apropriados.

6.3. Comunicação Assíncrona com Kafka

A comunicação assíncrona foi definida com Kafka para desacoplar o fluxo de pedidos do fluxo de pagamentos e permitir que o sistema continue respondendo mesmo quando um componente esteja temporariamente indisponível. Foram adotados os eventos obrigatórios definidos no desafio: pedido.criado, pagamento.aprovado e pagamento.pendente.

No sistema JavaEats_Fase3, o pedido e o pagamento possuem fluxos próprios, com isso, o processamento do pagamento já nasce no fluxo de criação de Payment, que depois chama o provider externo. Migrar esse gatilho para pedido.criado exigiria mudar o desenho funcional atual, e não apenas adicionar Kafka. Por esse motivo o pagamento-service não consome diretamente o evento, e o tenta processar usando processamento-pagamento-externo, mantendo o fluxo alinhado com ao comportamento esperado do sistema.

Além disso, a atualização do pedido orientada pelo evento pagamento.aprovado também não se encontra desenhada dessa forma, já que o status da Order já é determinado pela regra de domínio existente, baseada na soma dos pagamentos PAID em relação ao valor total do pedido. Implementar essa mesma decisão a partir do evento introduziria duplicação de regra e aumentaria o risco de inconsistência.

O fluxo operacional segue a lógica de eventos: quando um pedido é criado, o sistema publica o evento pedido.criado para fins de rastreabilidade e integração. Quando um pagamento é criado, o sistema tenta processá-lo como serviço externo. Se o pagamento for confirmado, o sistema registra a atualização e publica pagamento.aprovado. Se houver falha, o sistema publica pagamento.pendente, permitindo que o pagamento seja reprocessado posteriormente. Esse modelo mantém o sistema responsivo, reduz acoplamento em cenário de instabilidade e dá suporte ao reprocessamento automático sem bloquear o fluxo principal.

6.4 Resiliência (Resilience4j)

A integração com o processador externo de pagamento foi protegida com mecanismos de resiliência usando Resilience4j. A chamada ao serviço externo é tratada com Circuit Breaker e Timeout (e, quando aplicável, Retry), evitando que o sistema fique travado esperando respostas indefinidamente ou degradando por excesso de tentativas em cascata.

Quando o serviço externo falha ou se torna indisponível, o sistema aciona um *fallback* que mantém o pedido em estado consistente, marcando-o como pendente de pagamento e publicando o evento pagamento.pendente no Kafka. A partir desse ponto, a pendência entra em um fluxo de reprocessamento automático: um job/worker tenta

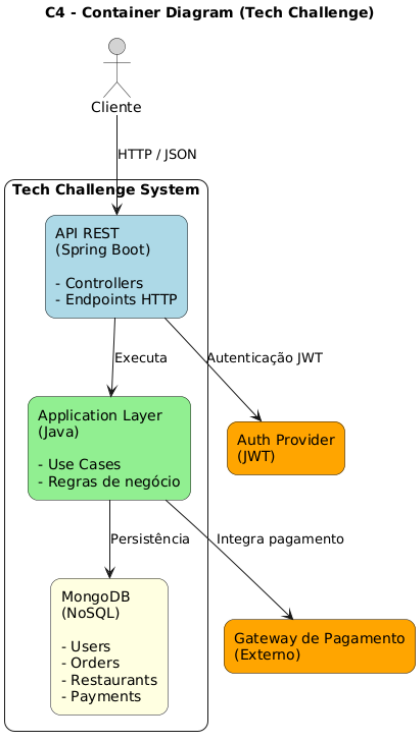
novamente processar os pagamentos pendentes em intervalos configuráveis (payment.reprocessing.fixed-delay-ms). Assim que o serviço externo volta a responder e o pagamento é confirmado, o sistema registra os metadados do processamento (por exemplo, provider e transactionId) e atualiza automaticamente o pedido para permitir a continuidade do fluxo de preparo, saída para entrega e entrega.

Esse desenho garante que falhas externas não quebrem o sistema nem interrompam o fluxo do usuário, ao mesmo tempo em que preserva rastreabilidade e consistência dos estados do pedido e do pagamento.

6.5. Boas práticas de arquitetura

O projeto foi desenvolvido seguindo boas práticas inspiradas em *Clean Architecture* / Hexagonal e separação por camadas, com responsabilidades bem definidas. A camada de *controllers* concentra os pontos de entrada (HTTP), os *use cases* implementam a lógica de aplicação e orquestração, o domínio concentra regras de negócio e entidades/objetos de valor, e a camada de infraestrutura contém *adapters* de persistência e integrações externas (MongoDB, Kafka e processador de pagamento). A figura 3 ilustra essa arquitetura.

Figura 3. Diagrama C4 da arquitetura do sistema



Essa organização reduz acoplamento entre regra de negócio e tecnologia, facilita testes e manutenção, e torna o sistema mais evolutivo — especialmente importante em um contexto de projeto acadêmico, onde a clareza da arquitetura e a justificativa das decisões fazem parte da avaliação final.

7. Configuração do sistema

7.1. Pré-requisitos

Para execução do sistema é necessário:

- Docker Engine instalado e em execução (Docker Desktop no macOS/Windows ou Docker no Linux);
- Docker Compose disponível;
- Acesso ao terminal;
- Estar posicionado na raiz do projeto, onde se encontra o arquivo: *docker-compose.yml*.

O ambiente Docker sobe os serviços necessários para a aplicação JavaEats_Fase3 funcionar corretamente. Isso inclui o MongoDB, o backend Spring Boot e o simulador externo de pagamento fornecido (procpag)

7.2. Execução

Após atender aos pré-requisitos, o sistema pode ser iniciado por meio do seguinte comando via terminal na raiz do projeto:

```
docker compose up -d --build --force-recreate
docker compose logs -f app
```

O primeiro comando cria e inicia os containers (reconstruindo a imagem da aplicação quando necessário). O segundo comando exibe os logs do serviço app, facilitando acompanhar a inicialização e identificar detalhes de configuração.
Após a inicialização, a aplicação estará disponível na porta configurada no arquivo docker-compose.yml (por padrão, http://localhost:8081).

7.3. Encerrar execução

Para encerrar a execução do sistema e remover os containers criados, pode-se utilizar o seguinte comando:

```
docker compose down -v
```

7.4. Observação sobre rebuild

Em caso de alterações no código-fonte, para garantir que os containers sejam atualizados com a versão mais recente da aplicação recomenda-se executar:

```
docker compose up –build
```

8. Execução do sistema

Com o ambiente em execução (Docker funcionando e API disponível), os testes do sistema podem ser realizados por meio da coleção de requisições (Postman/Insomnia) incluída no repositório, que já está preparada para conduzir o avaliador pelo fluxo completo de ponta a ponta.

Recomenda-se acessar o diretório docs/ (na raiz do projeto), onde estão armazenados os materiais de apoio do trabalho, incluindo os diagramas da arquitetura (ex.: C4 Model,) e os arquivos necessários para testar a API (*collection* e *environment*).

A coleção foi organizada para facilitar o uso por terceiros. O fluxo sugerido é iniciar pela pasta Setup (Create), que contém as requisições de preparação do ambiente de teste: criação dos usuários (CLIENT, OWNER e ADMIN), criação e vinculação de endereços, autenticação (geração de token JWT) e, em seguida, criação do restaurante e do menu.

Durante esse processo, os scripts do Postman salvam automaticamente os identificadores e tokens no *Environment* (por exemplo: tokenClient, tokenOwner, restaurantId, menuId, orderId), permitindo que as próximas requisições sejam executadas sem ajustes manuais.

Após concluir o Setup, o próximo passo é seguir para o diretório Orders, que contém cenários completos de negócio, organizados para validação do fluxo principal e exceções. De forma geral, os cenários cobrem:

- Fluxo de sucesso (pedido → pagamento → preparação → entrega);
- Fluxo de reembolso (pagamento reembolsado e impacto no pedido);
- Fluxo de recusa/cancelamento (quando aplicável).

As figuras 4 e 5 ilustram, respectivamente a organização das pastas de Setup do sistema e de pedidos com os 3 casos citados acima.

Figura 4. Organização de pastas mostrando o *Setup* da *collection*.

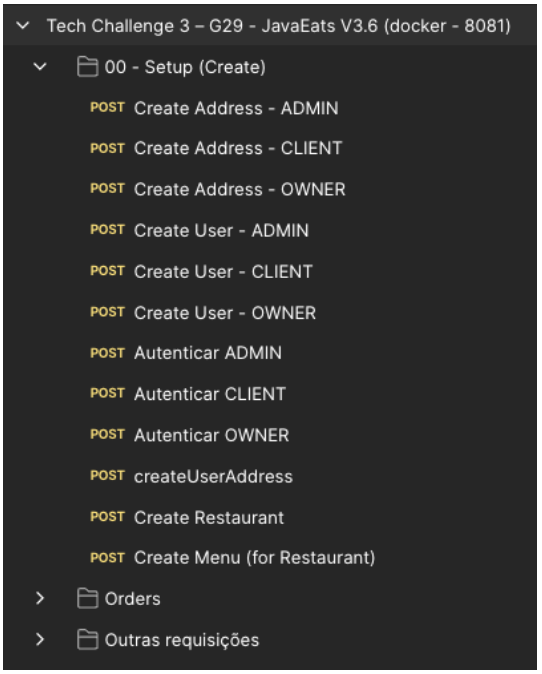
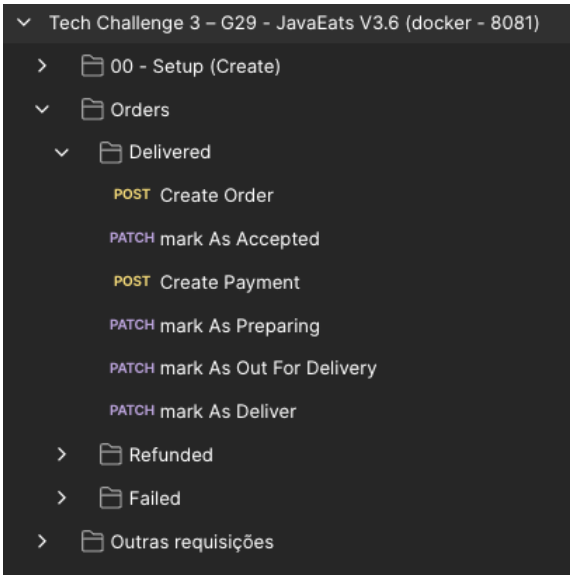


Figura 5. Organização de pastas mostrando os 3 casos de pedidos citados acima



Por fim, a pasta “Outras requisições” reúne *endpoints* complementares para consulta (GET), atualização (PUT/PATCH) e remoção (DELETE), úteis para validação pontual e auditoria do comportamento do sistema.